TRABAJO PRÁCTICO: GIT AVANZADO

Unidad 4 – Semana 1

Metodología de Sistemas II – Tecnicatura Universitaria en Programación a Distancia – UTN

Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26

Para algunas respuestas armé un repositorio de prueba y corrí los comandos, así las explicaciones tienen ejemplos reales.

Parte 1: La "física" de Git

1. El modelo de objetos

- **Blob:** guarda el contenido de un archivo, nada más. No guarda el nombre ni los permisos.
- **Tree:** representa una carpeta. Es una lista que relaciona nombres de archivo con sus blobs (y subcarpetas con otros trees), junto con sus permisos.
- **Commit:** apunta al tree raíz del proyecto y guarda los metadatos: autor, fecha, mensaje y el commit padre (o los padres, si es un merge).

Se relacionan en cadena: el commit apunta a un tree, el tree apunta a los blobs, y cada commit apunta a su padre. Así, cada commit es una **foto completa** del proyecto en ese momento. Lo comprobé en el repositorio de prueba:

```
$ git log --oneline
2f2d93f Agrega despedida
140806b Primer commit

$ git cat-file -t 2f2d93f
commit

$ git cat-file -p 2f2d93f
tree e2cbbd71940282cbb4447e71fa1dd41d5f6bf306
parent 140806bca61601c61c8076951077554c1eac0e47
author Ignacio Figueroa <ignacio@ejemplo.com> 1793739600 -0300
committer Ignacio Figueroa <ignacio@ejemplo.com> 1793739600 -0300

Agrega despedida

$ git cat-file -p e2cbbd7
100644 blob 69985126666f2a6e88398a0c34d598c69c1dc864       app.py

$ git cat-file -p 6998512
print("hola")
print("chau")

$ git fsck --full
(no muestra errores: la base de objetos está sana)
```

2. Direccionamiento por contenido

En Git cada objeto se guarda con un nombre que es el hash de su contenido. Por eso se dice que la "dirección" de un archivo es su propio contenido: para encontrarlo no se usa su nombre ni su ubicación, sino el hash de lo que contiene.




```
$ echo "hola mundo" | git hash-object --stdin
24db42bb7b999597a72801da70efd5876059bc0b
$ echo "hola mundo" | git hash-object --stdin     # mismo contenido, mismo hash
24db42bb7b999597a72801da70efd5876059bc0b
$ echo "hola mundo!" | git hash-object --stdin    # cambia 1 carácter
8087ff8cab9a2eeeedb63ff7ad08e63bdd121ed4
```

Esto sirve para dos cosas:

- **Integridad:** si alguien cambia un solo byte, el hash cambia por completo. Git lo detecta, porque el contenido ya no coincide con su nombre.
- **Deduplicación:** si dos archivos (o dos versiones) tienen exactamente el mismo contenido, tienen el mismo hash y Git los guarda una sola vez. Por eso hacer muchos commits no ocupa tanto espacio: los archivos que no cambian se reutilizan.

Aclaración: por defecto Git todavía usa SHA-1, como se ve en los hashes de 40 caracteres del ejemplo. SHA-256 se puede elegir al crear el repositorio (git init --object-format=sha256) y es más seguro, pero la idea de funcionamiento es la misma.

3. El grafo dirigido acíclico (DAG)

El historial forma un grafo en el que cada commit apunta a su padre. Es dirigido porque las flechas van de un commit a su padre, y acíclico porque nunca se puede volver a un commit anterior siguiendo las flechas.

Como el hash de un commit se calcula con su contenido, y ese contenido incluye el hash de su tree y el de su padre, si se modifica un archivo del pasado cambia su blob. Eso cambia el tree, que cambia el commit, y el commit siguiente, que tenía guardado el hash viejo como padre, también cambia. Así hasta el último. Un cambio mínimo hace que **todos los hashes de ahí en adelante cambien**.

Esto funciona como una **cadena de custodia criptográfica**: cada commit "certifica" a todos los anteriores. Si dos personas tienen el mismo hash del último commit, tienen garantizado que todo el historial es idéntico. Es imposible alterar algo viejo a escondidas.

Parte 2: Áreas de trabajo y flujo atómico

4. Las tres áreas

Git maneja tres áreas: el **directorio de trabajo** (los archivos que edito), el **staging area** o índice (lo que preparo para el próximo commit) y el **repositorio** (la base de datos .git con el historial).

- **git add:** Git calcula el hash del archivo, crea el blob en .git/objects y actualiza el índice (el staging area) para que ese archivo apunte al blob nuevo. Todavía no hay commit, pero el contenido ya está guardado.
- **git commit:** Git arma los trees a partir del índice, crea el objeto commit (apuntando al tree raíz y al commit padre) y mueve la rama actual para que apunte a ese commit nuevo.

Por eso el commit es **atómico**: se guarda todo lo que está en el staging o no se guarda nada. Además, el staging permite elegir exactamente qué cambios entran en cada commit.

5. El rol de HEAD

HEAD es un puntero que indica "dónde estoy parado". Normalmente apunta a una rama (por ejemplo main), y la rama apunta al último commit. Al hacer un commit, la rama avanza y HEAD avanza con ella.



HEAD está en estado **detached** cuando apunta directamente a un commit y no a una rama, por ejemplo con git checkout <hash> o al hacer checkout de un tag. Ahí se puede mirar y probar el código de ese momento sin tocar ninguna rama.

Para la **resiliencia** es útil porque permite volver a cualquier versión anterior para investigar un error (por ejemplo, ver desde qué versión aparece un bug, como hace git bisect), probar una versión publicada o compilar un tag viejo, sin riesgo de romper las ramas de trabajo. Si se hacen commits en ese estado, hay que crear una rama (git switch -c <nombre>) para no perderlos.

Parte 3: Gestión y cirugía del historial

6. Limpieza de historial

Para unir los 5 commits usaría un **rebase interactivo**:

```
git rebase -i HEAD~5

# En el editor dejo "pick" en el primero y cambio los otros por "squash" (o "s"):
pick   a1b2c3d fix 1
squash d4e5f6a test
squash 7a8b9c0 arreglo
squash 1d2e3f4 cambio
squash 5a6b7c8 cambio final

# Después escribo un mensaje profesional, por ejemplo:
# "Agrega validación de stock en el carrito de compras"
```

Otra forma más simple es git reset --soft HEAD~5 y después un git commit nuevo, porque los cambios quedan en el staging.

La diferencia entre las dos opciones:

- **git commit --amend** solo modifica el **último** commit: le cambia el mensaje o le agrega archivos que me olvidé.
- **El rebase interactivo** permite reescribir **varios** commits: unirlos (squash), reordenarlos, editarlos, cambiarles el mensaje o borrarlos.

Las dos reescriben el historial (crean commits nuevos con otro hash), así que solo hay que usarlas en commits locales que todavía no se subieron al remoto.

7. Manipulación quirúrgica: cherry-pick

Usaría git cherry-pick cuando necesito llevar **un commit puntual** a otra rama sin traer todo lo demás. Un merge, en cambio, trae todos los commits de la rama.

Ejemplo con un hotfix: en la rama develop hay muchas funcionalidades a medio terminar, y además se corrigió un bug crítico en el cálculo de precios (commit abc1234). Producción no puede esperar a que esté todo listo. Hacer un merge de develop a main llevaría a producción código sin terminar. Con cherry-pick llevo solo la corrección:

```
git switch main
git cherry-pick abc1234     # copia solo ese commit a main
git push origin main
```



Parte 4: Casos prácticos de resiliencia

8. Escenario de desastre

A) El trabajo no está perdido. git reset --hard solo movió la rama al commit anterior. El commit "borrado" sigue existiendo en .git/objects, solo que ninguna rama apunta a él y por eso no aparece en git log. Además, Git registra en el **reflog** cada movimiento de HEAD. Esos commits se eliminan recién cuando corre el recolector de basura (git gc), en general después de unos 30 días.

Hay que aclarar que si el trabajo **nunca se había commiteado**, ahí sí se pierde, porque reset --hard pisa los cambios del directorio de trabajo.

B) Procedimiento con git reflog:

- **1.** Ejecutar git reflog para ver el historial de movimientos de HEAD.
- **2.** Buscar la línea anterior al reset, que tiene el commit perdido (en el ejemplo, HEAD@{1}).
- **3.** Por seguridad, crear una rama que apunte a ese commit: git branch recuperado <hash>.
- **4.** Volver la rama actual a ese commit: git reset --hard <hash>.
- **5.** Verificar con git log que el commit volvió.

Lo probé en el repositorio de prueba:

```
$ git log --oneline
1f4f954 Agrega funcionalidad de la tarde
2f2d93f Agrega despedida
140806b Primer commit

$ git reset --hard HEAD~1      # el error
HEAD is now at 2f2d93f Agrega despedida

$ git log --oneline           # el commit ya no aparece
2f2d93f Agrega despedida
140806b Primer commit

$ git reflog
2f2d93f HEAD@{0}: reset: moving to HEAD~1
1f4f954 HEAD@{1}: commit: Agrega funcionalidad de la tarde
2f2d93f HEAD@{2}: commit: Agrega despedida
140806b HEAD@{3}: commit (initial): Primer commit

$ git branch recuperado 1f4f954

$ git reset --hard 1f4f954
HEAD is now at 1f4f954 Agrega funcionalidad de la tarde

$ git log --oneline
1f4f954 Agrega funcionalidad de la tarde
2f2d93f Agrega despedida
140806b Primer commit
```

9. Reset vs. revert

Por qué git reset es mala práctica en este caso: reset reescribe el historial, porque mueve la rama hacia atrás y "borra" commits. Si el commit ya se subió, mis compañeros ya lo tienen en sus repositorios. Para subir mi versión tendría que hacer un push --force, y el historial del remoto

quedaría distinto al de todos los demás. Eso genera conflictos, y alguien podría volver a subir el commit con error sin darse cuenta.

La alternativa segura es git revert <hash>. Técnicamente, revert no borra nada: crea un **commit nuevo** que aplica los cambios inversos del commit con error (lo que se agregó, se quita, y lo que se quitó, se vuelve a poner). El historial solo avanza, así que se puede hacer push normalmente y además queda registrado que hubo un error y que se revirtió.

Parte 5: Diagnóstico y auditoría

10. Comandos de bajo nivel (plumbing)

- **Para ver el tipo de objeto de un hash:** git cat-file -t <hash>. Devuelve blob, tree, commit o tag.
- **Para ver el contenido puro de un tree o un blob:** git cat-file -p <hash>. Para un tree también sirve git ls-tree <hash>.
- **Para verificar si hay objetos corruptos:** git fsck --full revisa toda la base de objetos y avisa si alguno está dañado o falta.

Todos estos comandos solo leen la base de datos, así que se pueden usar sin cambiar de rama y sin tocar el directorio de trabajo. Los ejemplos están en la salida del punto 1.